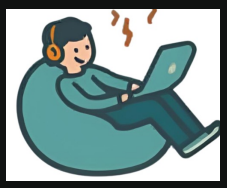

The Lazy Developer

06

MODULE

Authentication and Databases with Supabase (user accounts)

Learn what you need to do to setup your authentication, login accounts, and the associated security. Also learn about the database provided with Supabase and how you will use it. Lastly, we will cover the storage buckets that are also included.

Table of Contents

01. A quick overview of Supabase and why it's so important

10 min

Supabase at a Glance — The Non-Coder Tour

This first section is designed to ground you in what Supabase is, how much it costs to get started, and which three areas of its dashboard you'll revisit again and again while building in AI. By the end, the words "Auth," "Database," and "Storage" should feel as familiar as "Inbox," "Drafts," and "Sent" in an email client.

1.1 What Supabase Is (in plain English)

Supabase is a single online service that hands you three things every modern app needs without asking you to install or configure anything:

Supabase part	Plain-English role	Everyday metaphor
Auth	Handles sign-ups, log-ins, password resets, and "Who are you?" checks	A membership booth that issues key-cards
Database	Stores structured information—profiles, settings, records	A giant, secure spreadsheet
Storage	Keeps photos, PDFs, videos, and any other files	A labelled filing cabinet in the cloud

Because all three parts are already connected, it becomes considerably easier to implement a login system with account pages where each user's configuration is stored in Supabase and delivered to your app once you have a few key parameters setup in your environment variables.

> *AI prompt you could use right now*
> *"Explain what Auth, Database, and Storage would do in my app should we implement it."*

1.2 The Pricing Structure (and Why the Free Tier Is Usually Enough to Start)

Before you start building, it helps to know how far the free plan will take you. Supabase uses a tiered model; the table below shows the headline limits that matter most at launch.

Plan	Monthly cost	What it comfortably covers	Key limits (May 2025)
Free	$0	Learning, prototypes, personal projects	50 000 monthly active users, 500 MB database, 1 GB file storage, 5 GB bandwidth, up to 2 projects
Pro	From $25 per project	Early-stage production apps	100 000 MAU, 8 GB database, 100 GB storage, 250 GB bandwidth; usage-based overages after that
Team / Enterprise	$599 +	Larger teams that need SSO, SOC 2, on-call support	Custom limits & SLAs

For most learners, the Free tier is more than enough to build, demo, and even soft-launch an MVP. When your user base or data grows beyond those caps, upgrading is a one-click affair inside the billing tab.

1.3 The Three Dashboard Areas You'll Use on Day One

Every time you log into Supabase, you'll see a left-hand menu. Three items on that menu are so fundamental that nearly every AI prompt you write will touch at least one of them. The table outlines what each tab controls and why it matters, followed by a real-world parallel using Netflix:

Dashboard tab	Why it's mission-critical	"Netflix-style" example	Starter prompt for Cursor
Auth	No user management means no personalisation or secure data	Netflix must recognise you to resume the series you paused	"Set up sign-up and log-in screens that send a one-click magic link to the user's email."
Database	Holds the facts your interface displays and updates	Netflix logs every title you watch so it can make personalised suggestions	"Give me the SQL queries to create a table that records which shows each member has watched and rated."
Storage	Delivers the images, videos, or documents that bring data to life	Netflix stores poster art and thumbnails for every film	"Tell me where to add a private bucket for profile pictures that only the owner can access."

Taken together, these three services let even a small team deliver functionality that once required multiple specialised back-end systems:

1. Auth signs in Emma.
2. Database remembers Emma's watch-list.
3. Storage serves the cover images that make browsing feel rich.

Remove any one of them and the experience breaks down so we'll keep returning to this trio throughout the course.

FYI: Supabase is integrated into this very website. So all your settings, bookmarks, and account configuration are pulled from Supabase every time the page loads. AI knows its documentation very well.

2 Adding Sign-Up & Login

In this section you'll give visitors a way to create an account and come back later without ever touching a password field in code. You will:

1. Describe the sign-up flow in everyday language. Of just ask for a recommended one.
2. Feed that description to AI so it generates a working React component.
3. Wire the component to Supabase with environment-variable entries.
4. Explore alternative sign-in options (Google, GitHub, etc.) and see where new users appear inside Supabase's Auth %¸ Users screen.

AI will do all of this for you really easily.

By the end, you'll be able to log in to your own prototype exactly the way apps like Substack or Medium do—with a one-click magic link.

> *Before you start, you should create a 'dev' and 'prod' instance of Supabase for your app. As you have a dev setup running locally on your computer, that should connect to a dev instance in Supabase. Your production environment you deploy should connect to your production database. This is extremely imporant after you launch and have real users sitting in your database. You want to make sure your changes don't break things before you make any changes to your production environment.*

Universe	Lives on	What you do here
Development	Local machine	Experiment, break things, run migrations a dozen times, load dummy accounts
Production	my-app.com on Vercel (or your host of choice)	Serve real customers, store real profiles, keep strict Row-Level Security

Why bother with two?

- Safety net – Test a new RLS rule in dev; if it misbehaves, your paying users never notice.
- Clean data – Fake emails and lorem-ipsum recipes stay out of business analytics.
- Smooth upgrades – Run database migrations in dev first; when the tables look good, promote the same scripts to prod. Supabase's migrations and GitHub-Action workflow make this a one-click step.
- Preview magic – You dev server is going to show you how new features work after adding these configurations in the database. Once it's working just right, you can prepare the database migration to production. Or, you may decide not do pass on that change to production.

Step-by-step setup

1. Create two Supabase projects
2. Copy each project's keys (Settings %¸ API) and also make note of the database password that your create! You won't be able to see that again.

Client side:

- SUPABASE_URL
- SUPABASE_ANON_KEY

Server side:

- SUPABASE_SERVICE_ROLE_KEY for server-only code.
- DATABASE_URL for the connectivity string to your Supabase database

3. Store keys separately
4. Redirect URLs

With dev and prod clearly separated, you can keep iterating on your magic-link flow (and everything that follows) with full confidence—no accidental data wipes, no late-night rollbacks. As long as you have this well-documented in your readme file (ask Cursor to do that for you), you can reference this file in new chat windows so Cursor understands you have a dual setup and that you're working locally first to make any changes before they get applied to production.

2.1 Describe the Login Experience First

Before you tell AI what do to, picture the flow from the visitor's point of view and write it down in plain English.

> *Example*
> *"A visitor should click Join, type an email, and receive a magic link. When they click the link, they land on their personal dashboard If I'm missing anything, feel free to ask me any more questions to fill in the gaps."*

That is enough context for AI to scaffold the elements your front end needs. It may also ask you some additional questions about sign in options, and if you need the ability for a user to reset their own password.

2.2 Adding a simple sign in

Paste the flow you just wrote into AI and add one clarifying line that tells it what framework you are using.

> *AI prompt*
> *"Create sign-up and log-in screens that follow this flow: visitors click Join, enter their email, and get a magic-link email through Supabase. After they click the link, send them to /dashboard."*

AI replies with a React component that already imports @supabase/supabase-js, calls supabase.auth.signInWithOtp({ email }), and listens for the redirect. Under the hood, Supabase will send the email because any address without a password triggers the passwordless magic-link route.

You have the beginnings of a user login screen as part of this process but of course you will need to create a button/link somehwere on your app to take you to the sign in page.

Let's say that button is in the navbar, you should also consider what happens after you log in. Does the login button change to a "Dashboard" link? Does it change to a "Sign Out" button? Model this behaviour off of what you think works best for your users.

2.3 Add Two Environment Variables

The generated code needs to know which Supabase project to talk to and which public key is allowed in the browser. Both values live in the Project settings %¸ Data API panel:

Variable	Where to paste it	Why it belongs there
VITE_SUPABASE_URL	Client-side .env file so browser code can call Supabase	Tells the SDK which project to reach
SUPABASE_ANON_KEY	Same .env file	Public “anon” key that lets the browser talk to tables protected by

> *Tip for server work — If you will have a server role and you need extra privileges, place SUPABASE_SERVICE_ROLE_KEY only in server-side environment variables; never expose it to the browser. What does that mean?*

The /client folder has its own .env file and the /server folder has its own .env file. The service role key would go in the server's .env file. This key allows for access to the database bypassing the row level secruty we discussed in the pervious module.

After saving the file, restart your dev front end client and back end server so the variables load, then drop the component into a page. Open the browser, enter an email, and you’ll get a fresh magic link in your inbox.

2.4 “What Just Happened?” (Quick Peek Under the Hood)

1. Your prompt made AI call supabase.auth.signInWithOtp. When Supabase receives an email without a password it creates or finds that user and fires off a one-time URL.
2. The email contains a signed JSON Web Token (JWT). When the visitor clicks the link, the JWT is exchanged for a session cookie that your React app automatically stores.
3. The SDK now attaches that JWT to every request, which is why you can enforce Row-Level Security later without writing more code.

All of this happens without you touching passwords, resetting hashes, or maintaining sessions manually.

2.5 Adding Other Sign-In Methods

Magic links are great for low-friction onboarding, but Supabase also supports traditional passwords, one-time codes, and more than a dozen social providers (Google, GitHub, Microsoft, Apple, etc.).

To enable Google sign-in

1. In Supabase, open Auth %¸ Sign / Providers and choose "Google" and then go to paste your Google OAuth credentials. You can ask AI to explain to you where you get these from.
2. Update your component:

> *AI prompt*
> *“Tell me how to add a Continue with Google button to my existing login component.”*

Supabase will automatically merge the Google identity with any existing account that shares the same email.

2.6 Meet the Auth %¸ Users Screen

Open Auth %¸ Users in the dashboard and you’ll see a table of everyone who has signed up. From here you can:

- Deactivate or delete test accounts.
- Manually reset an email/password user’s credentials.
- Inspect custom metadata you attach later (e.g., subscription tier).

Whenever you test a new sign-in flow, refresh this screen; it confirms that Supabase created a real record and lets you debug issues like unverified emails or duplicate OAuth identities.

2.7 Build a very basic Dashbaord

Your /dashboard can be very simple to start. It might show some basic fields and allow you to make some changes to your account. This is a good way to test RLS. A user should only be able to see and update their own information and nobody else. They should be able to make minor changes to their own settings. For example, a name, contact information. Use your dashboard on this site as some inspiration!

3 Sending Those Emails Through Your Own SMTP Server (Brevo)

Nothing says “unfinished side-project” like a verification email from no-reply@supabase.com. Switching to your own SMTP service lets every message come from you@yourdomain.com, raises deliverability, and keeps branding consistent across marketing and transactional mail. Supabase makes the hand-off painless so you swap four numbers and a password in the dashboard and never touch your code again.

3.1 Why bother swapping the sender?

- Trust & Branding – A familiar “From” address reassures new users that the magic link is legit, not phishing.
- Inbox Placement – Reputable providers (Brevo, Postmark, Resend, Amazon SES) give you SPF/DKIM records that help emails land in Primary, not Spam.
- Rate & Volume Control – On Brevo's free tier you’re capped at ~300 mails/day; external providers scale to millions.
- Future Flexibility – Promotions, receipts, and passwordless links can all come from the same domain, using the same templates.

3.2 Choosing a Provider (Why We Demo Brevo)

Brevo’s free plan (300 transactional emails per day) is generous for prototypes or MVPs, and its dashboard is beginner-friendly. If you already use another service, the steps are almost identical so Supabase works with any SMTP host.

Provider	Free tier highlights	Notes
Brevo	300 emails/day	SMTP key lives under Transactional %¸ SMTP & API
Postmark	100 emails/month then $15	Excellent deliverability, region locking
SendGrid	100 emails/day	Larger learning curve, but popular
Resend	Pay-as-you-go	Modern API first, works via SMTP, too

3.3 Step-by-Step: Wiring Brevo to Supabase

1. Create a Brevo account
   Brevo will immediately nudge you to add and verify a sending domain.
2. Verify you own the domain
   Paste the DNS TXT & CNAME records Brevo shows into your domain host; click Verify when they propagate (usually a few minutes). Brevo can automatically update the domain name in Cloudflare for you! That makes things so much easier.
3. Grab your SMTP credentials
   In Brevo %¸ Click your Company on the top right %¸ SMTP & API, copy:

4. Paste those into Supabase
   Project Settings %¸ Authentication %¸ SMTP Settings %¸ Enable Custom SMTP > SMTP—fill host, port, user, password, and set “Sender email” to something like no-reply@yourdomain.com.
5. Save & send a test
   Supabase offers a Send test email button; enter your own address and confirm the message arrives. If it stalls, you can check the Supabase log details.
6. Customise the templates (optional but recommended)
   Under Auth %¸ Templates you can rewrite the subject line, add a logo, or localise copy for Confirm signup, Magic Link, Reset password, etc.

3.4 What changes in your code?

Nothing. The same React component you built earlier still calls supabase.auth.signInWithOtp(). Supabase reads the new SMTP settings and routes the email through Brevo instead of its default relay. Your .env may get updated with a SENDER_EMAIL entry but AI will inform you of that if it has added that extra step in the code.

For high-volume production apps, consider sending marketing campaigns through one email address and auth emails through another to keep reputations separate.

3.5 Quick Troubleshooting Checklist

Symptom	Likely culprit	Fix
“Connection refused” in Supabase logs	Wrong port or host	Use 587; check firewall rules
Test email flagged as spam	Missing SPF/DKIM	Wait for DNS to propagate, then retest
No email at all	SMTP password vs API key mix-up	Brevo: be sure you copied an SMTP key

---

3.6 When do you use the API key in Brevo?

You'll notice the ability to ustilise an API key in these settings too. This will come in handy if you want AI to code in the ability to send emails via a signup form to a contact list in Brevo. Then, you can craft marketing emails and send them to segments from this list. For example, a user newsletter signup form.

> *Prompt: I'd like a newsletter sign up form that captures the email addresses and sends them to a new contact list I've setup in Brevo. That list ID is 4 (the ID is required). Can you place that on the bottom of the homepage? Then work on the code and files for Breno integration. What steps do we need to work on first? Ask me any questions you have before starting any code changes.*

The API key usage is a server-side activity that warrnats grabbing the API key to put into your serer side .env file. This is not soething you want in your client .env file that a browser would use as anyone who captures that data sent mid traffic could potentially use your keys to send emails from your domain! This is of course where remembering to add some rate limiting logic in your code

would help htat regardless. Just ask AI!

4 Database Considerations When You Introduce User Accounts

As soon as an app lets people sign up, your data model changes in three significant ways:

1. Identity becomes the thread that ties every record together.
2. Personal preferences and private data appear often in very different shapes from user to user.
3. Legal and performance requirements tighten, because you are now storing information that belongs to real, identifiable humans.

This section helps you decide what to store, how to store it, and which default settings keep things fast, tidy, and GDPR-compliant, all before we lock the doors with Row-Level Security in the next module.

4.1 What actually goes in the database?

At minimum you will end up with two tables (or a table + JSON column) and a few supporting pieces:

Data bucket	Typical columns / attributes	Notes & options
Core profile	id (UUID from auth.users), email, created_at, full_name, avatar_url	Created automatically if you follow Supabase's profiles trigger pattern.
Preferences	Theme, language, notification toggles, etc.	If preferences vary wildly by user, stash them in a single JSONB column; Postgres lets you query inside JSON without predefined columns.
Subscription / billing	Plan tier, renewal date, stripe_customer_id	Keep PII-heavy billing data in its own table so "delete my account" can wipe it clean quickly.
Audit & activity	Last login, password-changed-at, consent flags	Helps with security reviews and GDPR "prove what you store" requests.

> *Performance tip – Add an index on any column you will search or join by, such as email or user_id. Supabase's Index Advisor in the dashboard will suggest missing indexes once query volume grows. Just talk more with AI about this.*

4.2 Designing the User Dashboard (what the user actually sees)

Every setting you capture should surface somewhere in the app so people can control their own data. A starter dashboard might include:

Section on the page	Typical fields	Why it matters
Personal info	Display name, avatar upload, time-zone	Lets users keep their identity current; avatar file itself lives in Storage, not the DB.
Appearance	Light / dark mode, font size	Stored as a JSON snippet like { "theme":"dark", "textSize":"l" }.
Subscriptions	Current plan, upgrade / cancel buttons	Mirrors data in your subscriptions table.
Email notifications	Marketing opt-in, "weekly digest" toggle	Store a boolean; read it before triggering any outbound email job.
Security	Change email, change password, MFA status	Some items (password hash) live in auth.users; others (MFA) in your own table.
Danger zone	"Delete my account"	Needed for GDPR Art. 17 right to be forgotten. ([GDPR][5])

At the top of this page in the navigation bar is the Account link. Click on that and see what settings we have here that you can control.

AI prompt you might use:

> *“Generate a page called /settings that loads the current user’s profile and preferences from Supabase, shows light/dark toggle, email notification checkbox, and a button to upload a new avatar to the avatars bucket.”*

4.3 Security & privacy implications (beyond RLS)

Security and hardening (through RLS) is paramount in everything you do. If you lost access to your database, your entire app is in the dust and you as a business lose all kinds of credibitlity. Think of all the data leaks that make it to the news! Those companies never fair well.

- Minimise what you store — GDPR Art. 25 requires “data protection by design and by default.” If you only need an email and a theme choice, don’t add birthplace or phone number.
- Separate PII from public data — keeping personal info in one table makes “export” and “erase” actions straightforward. It also avoids scattered foreign-key cascades when a user invokes the right to be forgotten.
- Encrypt sensitive columns — Supabase offers column-level encryption extensions; use them for anything you wouldn’t print on a postcard (API tokens, home address).
- Backups & retention — Supabase auto-backs up nightly. Decide how long you keep deleted-user backups in case you receive a legal request to purge old snapshots.
- Logging — mask or omit PII in application logs; stack traces can leak email addresses if you’re not careful.

4.4 Performance & storage head-room

Concern	Early-stage advice	When to revisit
Query speed	Index user_id, email, and any foreign keys you JOIN on.	When dashboards feel sluggish or table rows > 100 k.
Payload size	Keep big blobs (avatars, PDFs) in Storage, not as bytea columns.	Immediately if you see rapid growth in file uploads.
Table bloat	Use updated_at timestamps to let you archive old activity rows into cheaper cold storage.	Once activity table reaches millions of rows.
JSON vs many columns	JSONB is fine for < 20 keys per user and infrequent server-side filtering. Break out to a dedicated table if you need to filter/search individual prefs often.	Whenever preference queries require complex WHERE clauses or the JSON doc grows beyond ~8 KB.

Supabase’s Query Performance page will flag slow statements and recommend indexes automatically, saving you from premature optimisation.

4.5 Default project settings worth toggling on day one

Setting (Supabase %¸ Settings)	Suggested value	Why
Database time-zone	UTC	Avoids daylight-saving headaches; convert to local time in the UI.
pg_stat_statements	Enabled	Powers the Performance dashboard.
Weekly log retention	30 days for dev, 90 days for prod	Keeps storage costs predictable.
Auto-index advisor	Enabled	Surfaces missing indexes in one click.
Point-in-time recovery	Turn on for prod (cost involved)	Lets you roll back accidental deletes without manual dumps.

4.6 What's next?

You now have a profile table, a preferences strategy, and a dashboard map—all shaped with performance and GDPR in mind. In the next module we'll make sure only the rightful owner can read or modify those rows by turning on Row-Level Security and writing the first policy.

Share these links below with cursor to get some alternative talking points in the discussion and let AI present you reasons for and against them for your app.

-User Management | Supabase Docs

-Should I save user preference as JSON or individual columns?

-Storing user-defined segments JSONB vs separate table?

-Managing Indexes in PostgreSQL | Supabase Docs

-Art. 25 GDPR – Data protection by design and by default

-What does GDPR compliance mean for my database? - CockroachDB

-Ever ok to remove referential integrity on database design.

5 Keeping Each User's Stuff Private with Row-Level Security (RLS)

The moment more than one person can sign in, privacy stops being a courtesy and becomes a requirement.
Row-Level Security is Supabase's built-in way to say, in the database itself, "Alice may touch her rows, but never Bob's." Because the rule lives next to the data and not in your front-end code so it can't be bypassed by a rogue script or a forgotten endpoint.

> *Problem in one sentence*
> *"Alice shouldn't read Bob's secret chili formula."*

> *Plain-English rule*
> *"Only show a recipe to the person who made it."*

Here's a view of the RLS policies screen after they have been created:

Authentication %¸ Policies 'œ table-by-table list of who can SELECT, INSERT, UPDATE, or DELETE. You'll notice tiny padlock icons and natural-language labels like "Users can insert their own access logs."
That UI is a friendly wrapper around real Postgres policies that would be generated in Cursor such as:

```
create policy "Only owner can read"
on recipes
for select           -- applies to reads
to authenticated     -- logged-in visitors
using ( auth.uid() = user_id );
```

5.1 Turning RLS on using SQL queries

> *AI prompt a non-coder might give:*
> *"How do we enable Security for the recipes so that only the person who added a recipe can view, update, or delete it. Show me the steps and any buttons I need to click."*

You would simply copy and paste that and put it into the SQL editor an run it. Notice how the code has notes to explain what each part of the query does.
SQL Editor > Create a new snippet > Paste the code > Run

5.2 Testing that the lock actually works

Create an admin acccount - ask AI how to do this and how to enable this ad an administrator in the database.
Then create a user account - ask AI how to do this.

- Front-end sanity check – Log in as as both accounts and see if you have access to the differences between an admin and a standard user. Eg. An admin may have access to an admin control panel you've built, or access to see all data. A regular user shouldn't see any of that.

5.3 Performance & maintenance notes (quick hits)

These are good topics to ask AI about to determine when and if you need to worry about any of these.

Topic	What you need to know now	When to dig deeper
Indexes still matter	The filter auth.uid() = user_id uses the user_id column on every query, so be sure it's indexed. Supabase's Index Advisor will remind you.	When tables grow past ~100 k rows
Policy order	Multiple policies are OR-combined. If one says "admins can read everything," it overrides restrictive rules. Keep admin rules at the bottom for clarity.	As soon as you add a second role
Global reads	Analytics dashboards need wider access. Create a dedicated analytics role with its own relaxed policies instead of disabling RLS.	When you hook up BI tooling

5.4 Common pitfalls & quick fixes

This is you will put on your detective had and walk through all the pages and settings as an admin user and a regular user - and a non-logged in user! You're trying to break the system to see where the problems lie. Having the console window open in your browser while you do this is also really helpful so you can see any errors in the browser when trying to make changes or go to pages that should pull up information.

Symptom	Likely cause	Fix
Everyone gets empty results	RLS enabled but no policies	Add at least one SELECT policy
Guests can still read data	Policy set TO public instead of TO authenticated	Edit policy !' change role
Write works, read fails	You created an INSERT policy but forgot SELECT	Duplicate the policy and switch the action

Beta testing with some friends or people you know is always very helpful here. Have them go through the login process. Change passwords. Jump around your app.

Pro tip time!
Did you know, you could even ask AI to help you build a little help bubble at the bottom of each page that you click on and it pulls up a form to send emails so they can send any bug info. You can also have the form allow for screenshots to be attached so you can see what's happening! Now you have your Brevo integration, this is all possible!

6 Buckets: Supabase’s Built-In Storage and When to Use It

A bucket in Supabase is simply a named folder that sits on top of an S3-compatible store the Supabase team runs for you.
Inside that folder you can drop profile photos, PDFs, marketing images or entire video files and reach them through predictable, CDN-backed URLs. Because the storage service is already authenticated with the rest of your project, you get the same “Alice can’t see Bob’s stuff” guarantees you set up with RLS, but for files instead of rows.

6.1 How buckets work (public vs private)

Bucket type	Who can fetch the file once they have the URL?	Typical use-cases	Quick code pattern
Public	Anyone on the internet; URL contains no token	Blog post images, product screenshots, marketing assets	js const { publicURL } = supabase.storage.from('assets').getPublicUrl('hero.png')
Private	Only callers that supply a valid JWT or a short-lived signed URL	Avatars, invoices, pay-walled videos	js const { signedURL } = await supabase.storage.from('avatars').createSignedUrl(path, 60) (URL lasts 60 s)

Under the hood the URLs are served straight from Supabase’s CDN; reads do not count against your Postgres row quota.

6.2 Displaying images and videos in your app

Most image tags work out of the box:

```

```

Videos need two tweaks:

1. Serve with Range headers – Supabase already supports byte-range requests, so HTML5 video can stream/seek.
2. Pass a signed URL for private buckets:

Here's an example:

```
const { signedURL } = await supabase
  .storage
  .from('course-videos')
  .createSignedUrl('module-1/intro.mp4', 3_600);  // 1-hour TTL
```

If you embed many videos on one page, cache each signed URL in local state and refresh it only when it nears expiration (a common pattern to avoid extra round-trips). Just ask AI to help you implement this.

6.3 Good reasons to stay inside Supabase Storage

Reason	Why it helps non-coder teams
Single sign-on	The same JWT that RLS uses for tables governs file access so no extra identity access management (IAM) setups.
Flat pricing & free tier	Free projects include 1 GB of Storage; Pro starts with 100 GB and scales predictably.
Instant CDN	Files are fronted by a global CDN by default—no extra Cloudflare steps.
SQL joins	File metadata lives in Postgres, so you can JOIN images to rows for reports or cleanup scripts.
Zero-config CORS	The JavaScript client and Studio are pre-configured; uploads “just work” from localhost during dev.

6.4 When an external bucket (AWS S3, GCS, Azure) might be wiser

Scenario	Why an external store wins
Petabyte-scale video	You need Glacier/Tier-2 pricing or advanced transcoding pipelines.
Multi-cloud compliance	A client demands EU-only or GovCloud regions not yet offered by Supabase.
Ultra-fine IAM	Your ops team already manages granular S3 bucket policies and wants to keep a single access model.
Edge image transforms	You rely on CloudFront’s or Cloudflare’s real-time resizing features.

Hybrid is common: static images stay in Supabase for simplicity, 4K video offloads to S3 + CloudFlare for cost control.

6.5 Default settings to flip on day one

- File size limit – keep the default 50 MB while prototyping; raise it only when you test large uploads end-to-end.
- Storage policy – even in a private bucket, add an object-level RLS policy (owner = auth.uid()) so users can’t delete one another’s uploads. (Ask AI to create a policy for this with a simple prompt)

6.6 Quick checklist before you push to production

Question	Good-to-go answer
Does every private asset load through a signed URL that expires?	Yes
Are bulky files (videos, PDFs) outside your Postgres tables?	Yes
Do public assets set aggressive Cache-Control headers?	Yes
Have you measured bandwidth vs free-tier limits?	Yes

Once those boxes are checked, you can ship media-rich features with the same confidence you now have in your database rows.

And that's Supabase in a nutshell!

Here are some beginner-friendly resources you can explore right away:

- Supabase Docs – Getting Started
- Official Supabase Documentation Hub
- Supabase 101: Everything You Need to Get Started! (Video)
- Supabase Tutorial 2024: Complete Beginner's Guide (Video)
- Building a Simple To-Do App with Supabase & Next.js (Blog)
- Getting Started with Supabase (Blog)
- JavaScript Client Introduction – supabase-js
- Learn Supabase: Full Tutorial for Beginners (Video)